School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Utilising Multi Agent Path Finding to manage Unmanned Aerial Systems in urban environments.

Finlay Brydges

Submitted in accordance with the requirements for the degree of BSc Computer Science

2025/26

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (13/05/26)
Link to online code repository	`https://github.com/finlay-ab/uas-mapf-simulator`	(13/05/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Finlay Brydges

© 2025/26 The University of Leeds and Finlay Brydges

Summary

As the commercial use of Unmanned Aerial Systems (UASs) for tasks such as surveillance, infrastructure maintenance, and goods delivery increases, the density of urban air traffic will exceed the capacity of traditional human control centers. To ensure safety and efficiency at scale, an automated unmanned approach to UAS Traffic Management (UTM) is required, in which Airspace Service Providers (ASPs) negotiate flight plans with operators without continuous human oversight.

This project addresses that challenge by treating urban airspace management as a Multi-Agent Path Finding (MAPF) problem and evaluating a range of management policies through simulation. A custom discrete event simulation framework was developed in Python using SimPy, capable of modelling one or more ASPs serving dynamic flight plan requests across a configurable urban grid.

Eight policies were implemented and compared, ranging from uninformed search baselines (BFS, DFS) and a direct path policy that ignores obstacles and other agents to provide a lower bound reference on achievable flight time, through single agent informed search (A*) and a variant aware of dynamic occupancy (Occupancy A*), to prioritised and cooperative coordination strategies (Prioritised A*, Cooperative A*, and Windowed Hierarchical Cooperative A*). Each policy was evaluated under multiple demand levels and reservation configurations, with 5 independent random seeds per condition to support reproducible statistical comparison.

The evaluation produced three principal findings. First, the segment based reservation mechanism eliminates UAV collisions across the policies tested. However, it comes at a measurable cost in completion rate and average delivery time. Second, prioritised and cooperative methods outperform uninformed search baselines on safety and efficiency, but the dominant comparison between them depends on the demand level and reservation horizon rather than on the choice of search algorithm in isolation. Third, the completability of prioritised time expanded search is bounded its horizon rather than by available compute.

The results indicate that the principal driver of safe and efficient ASP-mediated UAS coordination is the presence of an explicit time aware coordination mechanism rather than the sophistication of the underlying single agent planner. Reservation based coordination is shown to be a practical and tractable alternative to full space-time MAPF for ASP mediated UTM. Directions for future work include the integration of coupled search methods such as Conflict Based Search, the evaluation of tactical reactive avoidance methods such as VOs, and the extension of the framework to three dimensional airspace.

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, Professor Leandro Soares Indrusiak, for his guidance, patience, and constructive feedback throughout this work.

I would also like to acknowledge the contributions of the open source software communities behind SimPy, NumPy, Matplotlib, and pytest, without which this project would not have been possible.

On a personal note, I want to thank my family and friends for their unwavering support and encouragement over the past seven months. I owe a special thank you to those who supported me during my recovery from a head injury this year.

Contents

1 Introduction and Background Research **1**

1.1 Introduction . . . 1

1.2 Literature review . . . 2

1.2.1 Single-agent path planning . . . 2

1.2.2 Multi-agent path finding . . . 3

1.2.3 UAS Traffic Management . . . 4

1.2.4 Reservation-based deconfliction . . . 5

1.2.5 Summary . . . 7

2 Methodology **8**

2.1 Requirements and design goals . . . 8

2.2 System Architecture . . . 9

2.3 Simulation Model . . . 11

2.3.1 Discrete event execution . . . 11

2.3.2 The UAV state machine . . . 11

2.3.3 The depot bound task model . . . 12

2.3.4 Hierarchical routing and gate handover . . . 13

2.3.5 Continuous time collision detection . . . 13

2.4 UAV management policy . . . 14

2.4.1 The policy abstraction . . . 14

2.4.2 The eight evaluated policies . . . 14

2.4.3 Policy instantiation . . . 15

2.5 Segment based reservation . . . 15

2.6 Project management . . . 16

3 Implementation and Validation **17**

3.1 Implementation overview . . . 17

3.2 Implementation of the simulation framework . . . 17

3.3 Implementation of planning policies . . . 18

3.4 Validation . . . 18

3.5 Engineering findings . . . 19

3.6 Summary . . . 19

4 Results, Evaluation and Discussion **20**

4.1 Experimental setup . . . 20

4.2 Effect of the reservation mechanism . . . 21

4.3 Computational cost . . . 22

4.4 Behaviour under varying conditions . . . 22

4.5 Algorithmic findings . . . 23

4.6 Limitations . . . 23

4.7 Conclusions 24
4.8 Future work 24

References **26**

Appendices **28**

A Self appraisal **28**
A.1 Critical self evaluation 28
A.2 Personal reflection and lessons learned 28
A.3 Legal, social, ethical and professional issues 30
A.3.1 Legal issues 30
A.3.2 Social issues 30
A.3.3 Ethical issues 30
A.3.4 Professional issues 31

B External Material **32**

Chapter 1

Introduction and Background Research

1.1 Introduction

The commercial use of Unmanned Aerial Systems (UASs) in urban environments is expanding rapidly, with applications ranging from goods delivery and infrastructure inspection to surveillance and sports coverage.

Industry projections and regulatory road maps anticipate that the density of low altitude urban air traffic will within the next decade exceed what can be co-ordinated by human Air Traffic Controllers (ATCs). This is why governments and regulatory bodies have developed frameworks for automated UAS Traffic Management (UTM): in May 2025, the European Union Aviation Safety Agency certified its first U-space Service Provider [3], and the United Kingdom Civil Aviation Authority has published targets aimed at enabling urban UTM by 2027 [21]. Despite this, the technical challenges of developing such an automated system and how it should perform, remains an area of active research.

A central part of the UTM problem is Airspace Service Providers (ASPs) must be able to co-ordinate the flight plan requests within a defined sector of urban airspace [4, 11]. Multiple ASPs are expected to operate at once in a networked system, managing the handover of UASs between sectors by following well defined protocols. The policies and protocols in which ASPs give authorisation decisions fundamentally shape the safety and efficiency of the UTM system, yet the comparative behaviour of different policies under realistic demand patterns is not well understood.

This project frames the problem of co-ordinating UASs through ASPs as a Multi-Agent Path Finding (MAPF) problem, the problem of finding collision free paths for multiple agents sharing a discrete environment [19]. MAPF is a well studied problem with a large body of literature on algorithms for safety, optimality and scalability of different automated management policies. As well as a substantial number of potential candidate algorithms ranging from simple heuristic searches to complex conflict resolving procedures. Treating the UTM problem as a MAPF problem makes it possible to evaluate policies systematically under quantified demand patterns to compare and contrast their performance (rather than judging them on intuition).

To support this evaluation, a custom discrete event simulation (DES) framework has been developed in Python using SimPy. The simulation models a two dimensional urban airspace divided into multiple grid sectors, each managed by an independent ASP and it is connected to its neighbours by directional gates that abstract the handover protocols described in the UTM literature. UAVs operate as lifelong agents (as introduced by Ma et al [12]), picking up jobs from designated depots and travelling between depots and destinations that may be located within their origin sector or in a different sector. Routing is structured hierarchically; a high

level planner that resolves the airspaces the UAV must pass through, and a low level planner which is responsible for finding a path within a sector. The low level planner is supplemented with optional segment based reservation mechanism, which provides a continuous conflict detection and can request the UAV to hold or re-route.

The aim of this project is to evaluate the comparative performance of automated airspace management policies for ASP controlled urban UAS traffic, with particular focus to the safety and efficiency trade off that distinguishes one policy from another. The associated objectives are to design and implement a multi-ASP discrete-event simulation framework, to define quantitative performance metrics covering both safety (separation violations) and efficiency (timeliness and throughput), to implement a range of management policies, and to conduct a systematic evaluation of these policies under varying demand patterns and constraints.

The remainder of this report is as follows. The remainder of this chapter reviews the literature on single agent planning, multi agent planning, UAS traffic management, and reservation based deconfliction. Chapter 2 describes the design and methodology of the simulation framework, including the architecture, the policies and the project management approach adopted. Chapter 3 documents the implementation, validation and testing of the simulation framework. Chapter 4 presents the experimental results and discusses their implications for ASP policy design. Finally Chapter 5 concludes the report and identifies directions for future work.

1.2 Literature review

This chapter reviews the relevant literature on which this project is based. The review straddles three research areas, algorithmic literature on pathfinding, operational and regulatory literature on UAS traffic management and the engineering literature on conflict resolution in multi-agent systems. The remainder of this chapter examines each and identifies the specific gap this project aims to fill.

1.2.1 Single-agent path planning

Single-agent path planning, the problem of computing a route between two points in a given environment subject to constraints (such as obstacles and cost functions), has been extensively studied as early as the 1950s and provides the foundation for the multi-agent path finding problem. This subsection introduces the algorithms that appear as components of the multi-agent path finding policies evaluated in this project, and the literature on which they are based.

Breadth-First Search (BFS) and Depth-First Search (DFS) are the most basic uninformed search algorithms for path planning [1]. BFS returns the path with the fewest steps from start to goal on a uniform cost grid, while DFS explores each branch to the maximum depth before backtracking. While both algorithms are complete neither is optimal, and both can be inefficient in large search spaces. They have been included in this project as a baseline for

comparison with more sophisticated algorithms and to provide a measurable benchmark for which more complex algorithms can be compared against.

The A* algorithm dominates the literature on single-agent path planning and is widely used in practice [6]. A* augments Dijkstra's algorithm with a heuristic estimate of the remaining cost from each point to the goal, and is guaranteed to return an optimal path whenever this heuristic is admissible (never overestimates the true cost). By focusing the search toward the goal rather than expanding all possible paths, it makes A* substantially more efficient than uninformed algorithms like BFS and DFS. While retaining the optimality guarantees. The implementation in this project uses the Euclidean distance (the straight-line distance) to the goal as its heuristic.

While the A* algorithm is optimal for pathfinding, it doesn't account for the dynamic nature of the environment, such as moving obstacles or changing conditions. This is where the D* algorithm comes in [18]. D* is an extension of A* that allows for replanning in dynamic environments. It maintains a cost map of the environment and updates it as new information becomes available, allowing it to efficiently replan paths when obstacles are detected or conditions change. Within the context of this project, D* lite is used which is an augmentation of D* that is more efficient and easier to implement [10] as it works backwards from the goal to the start making it more suitable for real-time applications (or simulations) as the end goal stays the same but the current state may change.

The final single agent path planning algorithm used in this project is a "greedy" algorithm. This algorithm always returns the straight line path between the start and goal, even if the path is not collision free. This algorithm is intended to provide a baseline for comparison with other algorithms, as it is the most efficient in terms of computational time but has no guarantees of optimality or safety. Therefore it is expected to have the shortest compute and commute times but the highest rate of collisions.

The single-agent path planning algorithms described in this section can plan one agent's path through an environment that is static, however they do not account for the presence of other agents within the environment. When multiple agents share an environment, and their plans interact, applying a single agent path planning algorithm produces plans that may conflict at execution time. Resolving these conflicts is the problem of multi-agent path finding, which is the focus of this project and is discussed in the next section.

1.2.2 Multi-agent path finding

Multi-Agent Path Finding (MAPF) extends the single-agent path planning problem to environments in which multiple agents operate and their paths must not conflict with each other. Stern et al. [19] define the MAPF problem as a set of agents, each with an assigned start and goal location, that must find paths to their respective goals without colliding with each other. The problem is typically modelled on a discrete grid or graph in which agents move to adjacent nodes at each time step. The standard objectives are makespan, defined as the time at which the last agent reaches its goal and flowtime, defined as the sum of individual

agents' path costs. The variant addressed in this work is non-anonymous, lifelong, and online: UAV identities are fixed, new flight plan requests arrive dynamically during execution, and each UAV operates a depot-bound task cycle in which it travels from its home depot to a pickup location, on to a delivery location, and back to its home depot before becoming available for a new job. This depot-bound structure is a constrained form of the lifelong Multi-Agent pickup and delivery formulation introduced for warehouse automation by Ma et al. [12], in which agents receive new tasks immediately upon completing a delivery; the constraint reflects the operational realities of UAS delivery, where drones typically return to a depot between flights for recharging and maintenance.

Prioritised planning, introduced by Erdmann and Lozano-Pérez [2], assigns agents a priority order and plans their paths sequentially, each agent treating the planned paths of higher-priority agents as dynamic obstacles. This approach is simple and efficient, and can be very effective in practice, especially when the priority order is well-chosen. However despite this method being substantially cheaper than the methods I will discuss below, it is incomplete. As higher priority agent's plan can block all remaining paths for a lower priority agent, even if a valid plan could be feasible given a different ordering. One way this is resolved, is by using Priority Based Search (PBS) [13], which achieves its completeness by searching over different orders when a conflict is detected.
Silver's Cooperative A* (CA*) [17] extends the A* algorithm to work with multiple agents by using an explicit time indexed obstacle representation called a reservation table. When an agent plans its path, it checks the table to ensure that its planned path does not conflict with the path of a previously planned agent. The same paper also introduced Windowed Hierarchical Cooperative A* (WHCA*), which only plans a fixed number of time steps into the future. This window based approach allows for more efficient planning as it reduces the size of the search space and by not planning the whole path it allows for less compute intensive replanning when the environment changes. However due to this limited planning it can lead to suboptimal paths.

The policies evaluated in this work are drawn from literature on uninformed search, single-agent informed and cooperative search. The full evaluation includes the full range of this spectrum, from simple greedy to WHCA*. The inclusion of greedy, BFS and DFS are to allow for a comparison with more sophisticated algorithms. The inclusion of A* and D* are to allow for a mid range comparison with more sophisticated algorithms, and to provide a measurable benchmark for which more complex algorithms can be compared against. CA* and WHCA* are the top of the spectrum and are the widely used algorithms in the MAPF literature.

1.2.3 UAS Traffic Management

The MAPF literature provides a range of algorithms in which multiple agents can be co-ordinated, but it does not address the specific context in which the project sits. The UAS Traffic Management (UTM) literature provides the framework for understanding the conventions, regulations and operational realities of UAS traffic in urban environments.

The idea of UTM was first proposed by Kopardekar at NASA [11] as a response to the anticipated growth of UAS operations in low-altitude operations. Kopardekar observed that current human controlled ATC infrastructure couldn't scale to the densities anticipated for commercial UAS operations. Therefore he proposed a new system of automated traffic management, in which multiple Airspace Service Providers (ASPs) would co-ordinate the flight plans of UASs within defined sectors of urban airspace. This proposal has since shaped the regulatory oversight and development of UTM systems. The FAA's UTM Concept of Operations [4] and the European Union's U-space framework [3] both define protocols outlined in the paper by Kopardekar, in which ASPs manage the airspace within their sector and co-ordinate with neighbouring ASPs to manage handovers of UASs between sectors. This literature provides the context for this project and the basis for the design of the simulation framework.

A growing body of research applying MAPF techniques to UTM has emerged in recent years. One line of research by Ho et al. [7–9] progressively develops a range of algorithms for co-ordinating UAS traffic focusing on decentralised execution joint with centralised planning. Another line of research by Sacharny and Henderson [15] focuses on a line-based architecture in which airspaces are restricted to predefined corridors. Not only does this make the problem more tractable but it also allows for segment based reservations which are more efficient than UAVs reserving individual grid cells as it reduces deconfliction from a high dimensional space time search to a one dimensional scheduling problem. This system is similar to the one adopted in this project, however the focus of this project is on the comparative performance of different policies for co-ordinating UAS traffic, rather than the development of a single novel algorithm.

Despite active research within the UTM literature, the comparative performance of different policies with their safety and efficiency trade-offs under realistic demand patterns is not yet well understood. Existing work tends to focus on either an optimal centralised solution and specific architectures rather than an empirical evaluation of candidate policies.

1.2.4 Reservation-based deconfliction

Conflict avoidance in multi-agent systems can be approached at two layers. At plan time, when agents commit in advance plans that are constructed to be conflict free, and at execution time, when agents adjust their motion reactively in response to an observed conflict risk. Operational UTM frameworks combine both with strategic deconfliction policies in planning and tactical deconfliction policies that can handle the uncertainty and dynamic nature that is inherent to real world UAS operations. [4]. This subsection examines the strategic mechanisms that motivate the design adopted in this project.

The reservation table introduced by Silver [17] is the fundamental mechanism for strategic deconfliction in MAPF. It indexes vertex time pairs that represent the current and planned occupation of a vertex at a given times. Agents can query the reservation table to check for potential conflicts with other agents, and can update the table when they commit to a plan. This mechanism is used in many MAPF algorithms, including the ones evaluated in this

project, as it provides a simple and efficient way to manage conflicts at plan time.

However, as the planning horizon increases (how far into the future agents plan), the reservation table and thus the complexity of finding conflict free paths increases substantially. Phillips and Likhachev [14] addresses this with safe interval planning by using "safe time intervals" to represent the time periods in which a vertex is free of conflicts, rather than using a reservation table that tracks the occupation of each vertex at each time step.

The reservation table is also limited as it relies on agents committing to plans in advance, assuming they will not change and as the planning horizon increases the complexity of finding and checking for a conflict free path increases substantially, and the risk of a future event changing the environment and invalidating the plan increases. This highlights the need for reactive deconfliction which will be discussed later in this section.

In the context of aviation, strategic deconfliction is conventionally achieved by storing a flight's trajectory in a flight plan. This plan is then used by ATCs to manage the airspace and ensure that flights maintain safe separation. This means that ahead of time flights are reserving segments of the airspace ensuring they will avoid conflicts by going in a route that always ensures a large enough separation with other flights. This is analogous to the reservation table but instead of reserving grid cells they reserve segments. This is more efficient and scalable as it allows the use of a continuous collision detection algorithm that checks segments for collision over time, rather than checking each grid cell at each time step. This approach is very similar to the lane based architecture proposed by Sacharny and Henderson [15], in which the airspace is divided into predefined corridors which allows for the deconfliction problem to be reduced from a high dimensional space time search to a one dimensional scheduling problem. The reservation mechanism implemented in this project follows along with the segment based reservation allowing for a more efficient and scalable strategic deconfliction.

Strategic deconfliction policies that use reservation tables are effective at managing conflicts at plan time, but they do not account for the uncertainty and dynamic nature of a real world environment. Factors such as weather, inaccurate flight time estimates, and unexpected obstacles can all lead to unplanned conflict points. This is why reactive deconfliction is required. It is a layer of conflict avoidance that operates while the agent is executing its plan, allowing it to adjust its motion in response to an observed conflict risk.

Furthermore, strategic deconfliction however, does not account for the dynamic and uncertain nature of real world environments, and thus reactive deconfliction is required to handle unplanned conflict points that may arise during execution. This is where a complementary body of work on reactive deconfliction comes in, it includes algorithms such as the Velocity Obstacles (VO) [5], which provide a method for agents to adjust their velocity in real time to avoid collisions with other agents. These algorithms can be used with strategic deconfliction policies and even simple single-agent path planning algorithms to provide a more robust and flexible approach to conflict avoidance. The framework in this project is capable of supporting a tactical layer through optional policies. However, the focus of this project is on the strategic deconfliction policies and thus the evaluation of reactive deconfliction policies is left for future

work.

1.2.5 Summary

The literature surveyed in this chapter provides the foundation for this project. It covers the three key areas of research in designing an automated traffic management system for UASs. However, it doesn't cover the specific questions that I want to answer.
MAPF literature has provided a range of sophisticated solutions ranging from reservation based search, conflict based search and line based architectures. Each of these solutions have been produced along side rigorous proofs guaranteeing optimality, completeness and scalability. However, the comparative performance of these policies under realistic demand patterns is not yet well understood. This project aims to fill this gap by providing a systematic evaluation of a range of candidate policies under varying demand patterns and constraints.

The collision avoidance literature has provided a range of solutions for reactive deconfliction (such as the VO family of algorithms), they can operate in complement with strategic planning however the methods are typically studied in isolation and thus the trade-offs between these policies when combined are not well understood. This project provides a framework that can support the evaluation of both strategic and reactive deconfliction policies, however the focus of this project is on the strategic policies and thus the evaluation of reactive policies is left for future work.

The gap that this project address sits at the meeting point of these three areas of research. The simulation framework developed in this project is designed to allow for a systematic evaluation of the trade-offs of using different strategic deconfliction policies under realistic demand patterns under a range of MAPF policies along with segment reservation deconfliction policies. The remainder of this report describes the design of this framework in chapter 2, its implementation and validation in chapter 3, and the results of the evaluation in chapter 4.

Chapter 2

Methodology

This chapter describes the design and methodology of the simulation framework developed in this project. Section 2.1 outlines the requirements and design goals that motivated the framework. Section 2.2 presents the architecture. Section 2.3 describes the model with a focus on the discrete-event approach. Section 2.4 describes the management policies and strategy pattern through which they are composed. Section 2.5 describes the segment-based reservation mechanism that forms the central experimental variable in the evaluation. Finally, Section 2.6 documents the project management approach adopted during development.

2.1 Requirements and design goals

The literature reviewed in Chapter 1 identifies a gap in the comparative empirical evaluation of management policies for ASP mediated UAS Traffic Management (UTM). Closing that gap requires a simulation framework that is sufficiently faithful to the operational context of UTM to make its results meaningful, and sufficiently flexible to support controlled comparison across a range of policies and operating conditions. This section sets out the requirements created with these considerations in mind, which together inform the design choices documented in the remainder of the chapter.

The functional requirements are as follows. The framework must model an urban airspace as a network of independent Airspace Service Providers (ASPs), each controlling its defined area and handling the handover of traffic to its neighbouring controllers. It must process flight plan requests dynamically during execution rather than as a fixed batch known in advance (to reflect the real world). It must support multiple management policies from MAPF reviewed in the chapter above. It must permit policies to be tested in the same environments and configurations. It must record sufficient data on each flight and simulation to support the safety, efficiency, and coordination metrics described.

The non-functional requirements are equally important. The framework must be deterministic given a fixed random seed, so that experimental results are reproducible. It must be modular enough to permit new policies to be added without modifying the simulation core, in support of both the comparative evaluations undertaken in this work and any future extensions. It must provide configuration through external files rather than parameters so experimental conditions can be varied without recompilation. It must complete experiments within a reasonable time on commodity hardware as it will be run many times in many configurations.

From these requirements, they derive several other design goals that recur throughout the rest of this chapter. The simulation must be discrete-event driven rather than time-stepped. Many considerations motivated this choice. First, performance: a time-stepped simulation processes every agent (UAV) at every clock tick regardless of whether its state has changed, while a

discrete-event simulation processes a UAV only when an event affects it, which scales considerably better as the number of UAVs and the length of the simulated horizon grow. Secondly, precision in collision detection: continuous-time geometric collision detection over analytic trajectory segments yields the exact moment and distance of closest approach, while a fixed-tick simulation can miss close approach events that occur entirely within a single time step unless the step is made impractically small. Third, reproducibility: a discrete-event simulation is fully determined by its random seed and configuration, whereas a time-stepped simulation introduces the time step itself as a hidden parameter whose value can affect experimental results. SimPy [20] is the natural choice for this paradigm in Python.

These requirements and design goals together motivate the architecture presented in Section 2.2, the simulation model described in Section 2.3, the policy abstraction documented in Section 2.4, the reservation mechanism developed in Section 2.5, and the project management approach summarised in Section 2.6.

2.2 System Architecture

The simulation framework is organised as a layered, modular architecture in which each layer has a clearly defined responsibility, relying on the layer below. The architecture illustrated in Figure 2.1 comprises these key layers: configuration, world structure, ASP, agents, planning, collision avoidance, and finally metric collection.

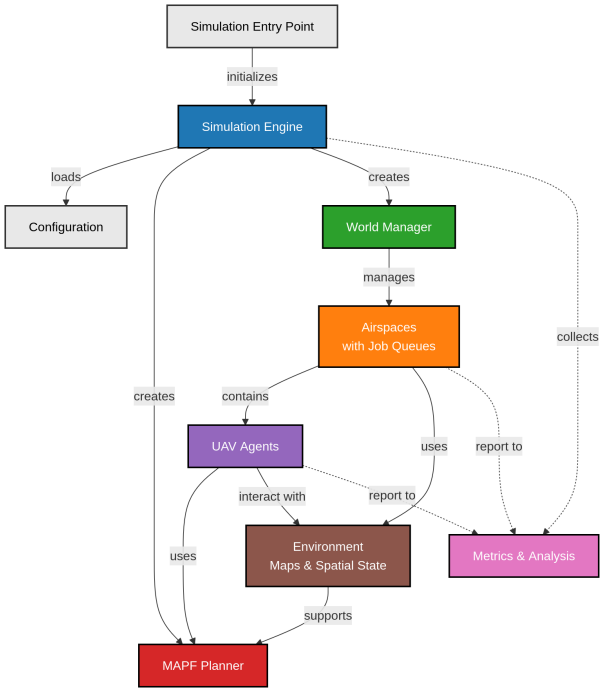

Figure 2.1: High level system architecture of the simulation framework. Solid arrows denote ownership relationships; dashed arrows denote runtime interactions.

At the top of the architecture is a single simulation object. It serves as the orchestrator and is constructed from the configuration file. It is responsible for instantiating the SimPy environment and the metrics collector, and creating the planning policies and the world

manager. Once instantiated, it delegates the running of the simulation event loop to a SimPy event manager; after this, it is only responsible for recording the final metrics to a file on completion.

The world manager owns the world structure and manages the inter-airspace coordination. It loads a world manifest that specifies the layout of airspaces and the connectivity between them. From the given manifest it also initiates an Airspace object per sector. Once the airspaces have been created, they are represented as nodes in a graph and the gates are added as edges. This is what is used when a UAV requires a route that crosses airspace boundaries, and it returns a sequence of airspaces and gates that the local planner within each airspace uses to get the UAV from its current position to a gate.

Each Airspace represents one sector of the simulated airspace. It owns per-depot job queues populated by job generation processes (described in Section 2.3). A set of SimPy resource objects representing depots and gates as capacity-constrained resources. The fleet of UAVs whose depot is within the sector. Each airspace also instantiates an ASP (Air Service Provider) object that manages the sector's dynamic state from tracking UAV position to continuous collision detection through segment reservation from trajectories (described in Section 2.5). The static spatial structure of the environment (obstacles, geometry, and depots) is held in the GridMap; together these comprise the environment layer in the architecture. Both the static and dynamic environments help implement the flight plan management role that UTM literature attributes to a service provider [4, 11].

UAV agents (which belong to airspaces) each run a SimPy process with their own state machines. A UAV holds a reference to its current management policy, its home depot's job queue, and its parent airspace along with a set of configurations loaded from a configuration file including but not limited to maximum speed, safety radius, and acceleration. When idle, a UAV yields until its home depot has a free job it can pick up. When a UAV picks up a job, the state machine kicks into effect. It requests a route through global airspace which returns a sequence of waypoints including the start point, gates to other airspaces, and the destination. Then the UAV will register with the ASP to query for predicted collisions and update its own trajectory over time. The full lifecycle of the UAV and its eight-state machine will be described in Section 2.3.

The MAPF planner is an abstraction that every policy fits behind. Every policy must inherit the common interface which allows for easy plug-and-play in the configuration. This means that Greedy, BFS, DFS, A*, Occupancy A*, Prioritised A*, Cooperative A*, and WHCA* all inherit velocity calculators and can return a list of waypoints. The Factory classes are able to be given a policy identifier from the configuration and instantiate the corresponding policy. This decoupling is essential for the comparative evaluation reported in Chapter 4, since it allows policies to be exchanged without modification to the airspace, UAV, or metrics subsystems.

The metrics subsystem records per-flight, per-collision, and per-run data during simulation and writes the results to CSV. Configuration is loaded from external JSON files and can be

modified by sending a config.py data object or a command line argument when running main. Externalising the configuration this way allows for testing without changing the source code, such as when giving the simulation a variety of seeds, run times, policies, and other configurations evaluated in Chapter 4.

The choice to model the simulation as a discrete-event-driven system, rather than as a time-stepped one, is fundamental to the architecture and is justified in Section 2.1. At the architectural level, this choice is reflected in the use of SimPy as the simulation core. This is because SimPy processes are routines that can yield control to the event loop at decision points, producing a simulation in which simulated time advances directly between events rather than at fixed clock intervals. This in turn allows continuous-time collision detection within the ASP, removing the time quantisation issues as dt would impact collision detection.

2.3 Simulation Model

This section describes the dynamic behaviour of the simulation; how UAVs behave over time, how jobs are generated, how routes are produced, and how conflicts are detected and resolved. This will explain how the static architecture in Section 2.2 provides a structure in which all of the simulation takes place.

2.3.1 Discrete event execution

The simulation uses SimPy at its foundations, treating each UAV, gate, and job generator as a yield-control or decision point in an event loop. This means simulated time advances directly between events; therefore, when there is no UAV making a decision, the event loop jumps forwards to the next scheduled event such as a UAV reserving a new segment once completing its last jump or a new job getting generated. These are the principles behind discrete-event-driven simulation.

There are two key implications of this approach. Firstly, there is no fixed simulation tick, meaning no dt parameter that controls the granularity of state updates; adding one would not change behaviour because the simulator processes events at the exact time they occur. Secondly, two events at the same point in time are ordered deterministically by SimPy's internal event priority mechanism. This means that given the same seed, if you ran the same simulation configuration, it will give the exact same solution.

2.3.2 The UAV state machine

Each UAV is a SimPy process whose actions are determined by which state it is in at a given point. UAVs have eight states. The states and their transitions are illustrated in Figure 2.2.

A UAV begins in the `IDLE_DEPOT` state, waiting at its home depot for a job to be placed into the depot's queue. The UAV will pick up the job and request the global (inter-airspace gate locations) route and local (current airspace) route. Then it transitions to `TAKEOFF`. During

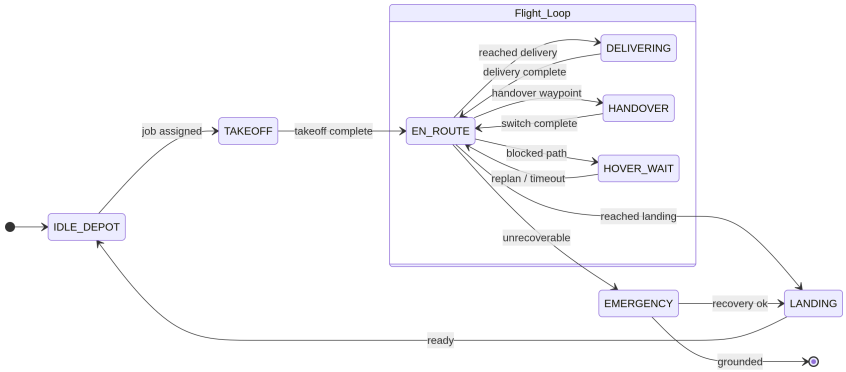

Figure 2.2: UAV state machine. Solid arrows indicate state transitions; arrow labels indicate the triggering conditions.

`TAKEOFF`, the UAV reserves a depot slot through a SimPy resource, moves into a takeoff position, and waits out a configured takeoff delay. When the delay elapses, the UAV transitions to `EN_ROUTE`.

The `EN_ROUTE` state is where the UAV does most of its work, handling the actual flight between waypoints. From here, it can shift into five different modes depending on the situation. If it reaches a delivery spot, it switches to `DELIVERING` to wait out a set drop-off time before heading back to `EN_ROUTE`. When crossing into a new sector at a gate, it enters `HANDOVER` to update its coordinates and airspace references; the system stays active during this move so it can seamlessly return to its flight in the new context.

Once the UAV reaches the end of its trip, it transitions to `LANDING` to grab a depot slot and finish its approach, eventually ending up in `IDLE_DEPOT` for the next job. If the path gets blocked, it enters `HOVER_WAIT` to take a breather and try to replan its route. If that replan works, it gets back to flying, but if the UAV completely runs out of options or cannot recover its progress, it shifts into `EMERGENCY` mode.

The `EMERGENCY` state acts as a fail-safe for when the system hits a wall it cannot plan around. When this happens, the UAV tries to limp back to its home depot. If it is already close enough, it just switches to `LANDING` and finishes the job like a normal flight. If it makes it back successfully, it ends up in `IDLE_DEPOT`, ready to work again. However, if the recovery fails over and over, the UAV eventually just grounds itself right where it is and stays in the `EMERGENCY` state.

This setup is key for testing the whole fleet. Without this "fail-safe" logic, a single stuck UAV could crash the entire simulation or mess up the data. By isolating these failures, one broken flight does not stop the rest of the fleet from finishing their missions.

2.3.3 The depot bound task model

Tasks are generated independently at each depot by a SimPy process whose interarrival times follow a Poisson distribution. This choice was made as it reflects that the delivery requests

arrive independently of one another from a consistent source, and follows the convention adopted in UTM studies such as Ho et al. [9]. Furthermore, by using this single number variable it allows for it to be easily modified to adjust demand levels in the evaluation. When a task is generated, a destination is sampled and checked for validity (if there exists a path from the depot to the delivery); tasks that cannot be planned are discarded and have no relevance in metrics.

2.3.4 Hierarchical routing and gate handover

A UAV that needs to travel from one Airspace to another must cross one or more gates. Routing is structured hierarchically to handle this. When a UAV gets its task, it plans its inter-airspace route by consulting the World Manager. The World Manager, through its graph of how airspaces relate to each other, returns a list of waypoints which outline the start, gates and their location, and end.

Within each airspace, the UAV's policy plans a low-level path. The low-level planner is only given two points within the current airspace: where the UAV is and where it is going within that airspace, be it a depot, gate, or delivery location. This separation of high-level and low-level planning means that the local planner only needs the GridMap and the global planner the inter-airspace graph.

Gates between airspaces are represented as SimPy resource objects with a capacity of one (modelling the requirement only one UAV may occupy a given gate cell at a time). When a UAV knows at the end of its next movement it will reach a gate, it will request that resource. If the gate is currently being held by another UAV, it will wait at a given waiting point. Once the gate is acquired, the UAV transitions to the `HANDOVER` state and its airspace and therefore its ASP is updated to the new sector. Completing the handover, the UAV switches to `EN_ROUTE` to plan its next steps.

2.3.5 Continuous time collision detection

The ASP performs collision detection on UAV trajectories continuously rather than at given time step intervals. When a UAV requests a new path or advances along an existing path, the ASP evaluates the given UAV's trajectory against other UAV trajectories. It calculates the linear separation between the pairs of vectors and if this is within the radius of the UAVs, then they will collide if the segment is allowed to be executed.

Continuous-time collision detection complements a discrete-event-driven simulation as it does not require fixed ticks and removes the inherently flawed collision calculations those provide. Given a large enough time step, one can miss near collisions or even collision events entirely, as it may skip over the exact point the UAVs are closest to one another.

2.4 UAV management policy

The simulation framework supports a family of management policies that share a common interface, which allows them to be switched out within the configuration to allow for evaluating them under the same conditions. This section describes how the policies are abstracted, how the policies are evaluated, and the mechanism by which they are instantiated.

2.4.1 The policy abstraction

All policies share a common abstract base class `MAPFPolicy`, which defines the operations every policy must support. The primary method is `plan_path(start, goal, context)`. It returns a sequence of waypoints connecting to the start of the goal (subject to constraints such as WHCA* can return partial somewhat optimal paths which is its strength). The secondary method is `get_velocity(uav, target)`, it returns the velocity at which the UAV should advance towards its next waypoint. By decoupling ASPs and UAVs from policies it ensures that the observed effects of changing a given policy are from difference in policy and not underlying code differences in state machines or elsewhere.

2.4.2 The eight evaluated policies

The eight policies span the three categories of search, unconstrained and uninformed, single-agent informed, and multi-agent informed cooperative methods.

Greedy is the simplest baseline. It performs no search, it just returns the start and end as waypoints, without even a collision check. Greedy was included as the lower bound for computation and job completion time and the higher bound for jobs completed.

Breadth-First Search (BFS) and Depth-First Search (DFS) are the two uninformed search baselines. BFS returns the path with the fewest grid steps from start to goal; while this does return the shortest path, its method of exploring every grid cell around it, expanding the search by one cell at a time, means it returns a highly computationally inefficient result. DFS has likewise been added for comparison; however, as it searches to the maximum depth before backtracking, it should result in very inefficient paths.

A* is the single-agent informed search baseline. It performs a weighted shortest path search over the grid using Euclidean distance for its heuristic.

Greedy, BFS, DFS, and A* are all unaware of the other agents within the environment so they should also give an insight into how much multi-agent planning can reduce collisions.

Occupancy A* extends A* with the awareness of other agents' current positions; before planning, it gets the dynamic blocked set by putting temporary obstacles in the grid map where each UAV's radius covers. Occupancy A* occupies the midpoint between a full MAPF solution and single-agent solutions. Its greatest setback is that as soon as the plan is made, it

is likely almost all the UAVs will have moved from their current points to new ones, and the algorithm does not update or account for this.

Prioritised A* is the first of the three reservation-based cooperative policies, drawing on the time-expanded reservation table of Silver [17]. Each agent plans using a variant of A* in which search nodes are aware of the time in which a cell has been reserved as instead of just reserving their current cell they stamp their whole route into a reservation table. Future agents then must not reserve the same cells at that given time. Within the framework of this project, I have added different priority modes such as hash-based or first-come, first-served for agent planning.

Cooperative A* is a configured variant of Prioritised A* with two key differences: lexicographic priority ordering rather than hash-based and an extended goal hold duration. The goal hold duration increase is key to preventing a recently arrived agent from being immediately displaced from its goal cell. While it shares the same underlying code as Prioritised A*, they differ in configuration which can produce different results in the evaluation.

WHCA* (Windowed Hierarchical Cooperative A*) adds a window mechanism to the reservations on top of Prioritised A*. When an agent plans, it only reserves the cells within a given time window. This means it returns a partial plan which is optimal within the window but may not be complete. This however does allow for more flexible plans as UAVs can replan more often and adapt to the dynamic environment.

2.4.3 Policy instantiation

Policies are produced by a factory class which takes a policy identifier from the configuration file and returns the corresponding policy object. This decoupling allows for policy choices to be hidden by the factory and for new policies to be easily added without modifying the rest of the codebase. This is key for the comparative evaluation in Chapter 4 as if policies were not to be decoupled you may accidentally change core simulation code that could affect the results and make it less clear that the differences are from the policy itself.

2.5 Segment based reservation

The reservation mechanism is a key variable in the evaluation reported in Chapter 4. It extends the reservation table approach of Silver [17] from discrete cell-time pairs to segment-based reservations over intervals of a given velocity. A reservation commits a UAV to a trajectory segment over a continuous velocity, time window, and its time of occupation.

The implementation in the framework of this project is further unlike a discrete reservation table, as an overlap between two reservations is not in itself a conflict. When a new reservation overlaps an existing one in space, the ASP performs a continuous-time check along the space between the two points given to determine whether the two UAVs will be present in the overlapping region simultaneously and rejects the new reservation only if both spatial and temporal overlap are confirmed. This is conceptually a coarse-grained analogue of the safe

interval reasoning in SIPP [14], in which the same insight (that a vertex's occupancy varies over time and can support multiple agents at different times) is exploited to compress the search state space. The continuous-time variant of this insight removes the artefacts of fixed step reservation tables and is what allows the mechanism to operate over vectors and not grids. Furthermore, this approach is more efficient within a discrete-event simulation as it doesn't require a constant timestep to check for conflicts in the background.

When a UAV plans under a reservation-aware policy (Prioritised A*, Cooperative A*, or WHCA*), the ASP requests existing reservations, allowing these reservation-aware policies to plan around any such conflicts. Non-reservation-aware policies (Greedy, BFS, DFS, A*, Occupancy A*) only passively interact with the reservation system, as they will update it with their plans but do not query it (allowing for conflict checks to be made by the ASP for metric collection).

The `allow_predicted_collisions` parameter controls the simulation's response when a predicted conflict cannot be resolved by replanning. When `True`, the simulation permits the UAV to proceed and records any collision. I.e., the reservation table is advisory rather than binding. When `False`, the UAV transitions to `HOVER_WAIT` and attempts to replan, escalating to `EMERGENCY` if repeated replans fail. The comparison between these two configurations is the central experimental variable of Chapter 4, isolating the contribution of strategic deconfliction by holding the underlying search algorithm constant and varying only whether reservations are enforced.

2.6 Project management

The project was developed over approximately seven months using an iterative workflow supported by GitHub Projects, GitHub Issues, and GitHub Actions. This section summarises the development methodology, the version control practice, and the testing infrastructure that supported the work. The codebase was managed in a single Git repository hosted on GitHub. All modifications were made on branches rather than directly on the main branch. This allowed changes to be reviewed and tested before being merged, meaning that the main branch would always store a stable example.

GitHub Projects was used to organise issues; they were organised into five columns (Backlog, Ready, In progress, In review, Done). They were further grouped into iterations representing approximately monthly milestones. Each issue carried estimates of size and priority allowing for better day-to-day planning of what needed to be done. Furthermore, having milestones, they would outline the general roadmap providing a longer-term plan against which iteration progress could be assessed. Automated testing was configured through GitHub Actions. This allowed the test suite to be run each time a branch was merged giving an immediate signal to stop merging and to see what broke.

Chapter 3

Implementation and Validation

This chapter describes the implementation of the simulation framework outlined in Chapter 2 and the validation that has been implemented. Section 3.1 gives an overview of the codebase. Sections 3.2 and 3.3 describe the implementation of the simulation and the management policies. Section 3.4 documents how validation was performed. Section 3.5 documents findings from development that informed the implementation. Finally, Section 3.6 summarises the chapter.

3.1 Implementation overview

The framework was implemented in Python 3.12 with approximately 2,800 lines of code distributed across modules outlined in Chapter 2. SimPy was used as the core of the discrete event driven simulation and NumPy for numerical operations. The codebase is organised hierarchically based on the area of function a class serves. At the top level in the root code directory are `simulation.py`, `main.py`, `factory.py`, and `config.py`. The root also contains a policy package with the eight policies and an environment package with the airspace, ASP, and grid map.

This implementation follows the architectural design of Chapter 2, with a factory that can select any planning policy that follows the `MAPFbase` class and the separation of ASP and Grid Map within the Airspace module. The rest of this chapter describes the implementation of the core simulation framework and the validation used. It goes on further to discuss the challenges encountered and the design choices made to overcome them.

3.2 Implementation of the simulation framework

The simulation core is built around three classes which are SimPy processes. Firstly, UAV processes that drive the state machine described in Section 2.3, secondly, job generator processes that produce jobs at a given rate, and finally, the simulation orchestrator that constructs the SimPy environment and world.

For each UAV there is a SimPy process. This means that once the UAV's run method is called, it is registered and will run until terminated; as UAVs are lifelong, this means when the simulation finishes. It is a process that yields control to the SimPy event loop at different decision points. These include waiting for a job, taking off, landing, and waiting for a gate. They also include simulating movement between waypoints, in which the UAV yields until the simulation clock allows the UAV to have "reached" the next waypoint. It is important to note that while the UAV is moved between waypoints and not in small steps, the collision detection

method used does not rely on this, and only the segment between the two points and the time it will take are needed to check for collisions.

Jobs are generated by a simple process that loops indefinitely. It yields a random time as defined by the Poisson distribution, and then once that time is up, it will generate a job, add it to the queue and metrics, then repeat by generating a new yield time and waiting.

3.3 Implementation of planning policies

Many of the eight planning policies share a substantial amount of code. A* is foundational and is extended by Occupancy A*, Prioritised A*, and WHCA*. Occupancy A* extends A* by overriding the obstacle set to include other UAVs' current positions. Prioritised A* extends A* by adding time expanded search nodes and a reservation table. Cooperative A* does not extend A* but instead extends Prioritised A* by fixing the priority ordering and adding goal holding. WHCA* extends Prioritised A* by adding the windowing mechanism by which partial paths can be returned.

BFS, DFS, and Greedy are independent implementations rather than A* derivatives. BFS uses a FIFO (first in first out) queue, DFS uses a LIFO (last in first out) stack, and Greedy bypasses search entirely, returning only the start and goal positions as waypoints.

One detail from the implementation of reservation based policies is worth mentioning. The table is stored as two dictionaries: a vertex table keyed by (cell, `time_index`) and an edge table keyed by (`from_cell`, `to_cell`, `time_index`). The edge table is required to catch UAVs swapping cells at the same time, and the vertex table handles the segment safety system described in Section 2.5.

3.4 Validation

The implementation of the framework was validated through a combination of three methods. A unit test suite was developed to test the most critical components such as the A* policy and collision detection. A full simulation run with policies being run multiple times under the same configuration was used to verify the determinism of the simulation. Finally, end to end testing was performed through collecting metric data and log files and checking for expected elements and data.

The project includes a pytest suite containing 28 tests across 10 test modules, covering policy behaviour, ASP, physics, metrics, state machine logic, and end to end output generation. Coverage, which was measured with a tool that logs which lines of code are executed during testing from the pytest suite, logged 33% coverage across the source code root folder. Some key components such as the A* and Greedy policies received over 80% coverage. Lower coverage is observed in code that is harder to test in isolation, such as the UAV process in `entities.py` (10%) and the airspace orchestration in `airspace.py` (0%). However, that is not to say they were not tested. The full end to end test suite and the experimental results in

Chapter 4 provide substantial coverage of the whole codebase, including the components that are harder to test in isolation.

Testing not only prevents unexpected errors and crashes, but it also provides confidence in the correctness of the implementation. Examples include testing if changing the connectivity in A* produces the expected change in path, or ensuring that collision checks, which are so fundamental to the evaluation in Chapter 4, are correctly identified.

The determinism verification is a critical part of the validation process. As the simulation was designed to be fully deterministic (given a fixed seed) to support reproducibility of the experimental results, it was essential to verify this property. To do this empirically, every experimental configuration was run twice with identical inputs and the resulting output files were compared bit for bit.

Integration testing was performed by running the full simulation and checking the output file for the expected elements. This provides confidence that the metrics being collected are the same as the ones being logged. Continuous integration was achieved by using GitHub Actions to run the unit tests on every pull and push request to the main branch. This provided confidence that when changes were made to the codebase, the core functionality was not broken.

3.5 Engineering findings

A couple of engineering findings emerged during development that meaningfully informed the implementation.

The Greedy policy required explicit static object detection, as despite the policy producing straight line trajectories that frequently pass through objects (unlike other policies which avoid objects), it reported zero collisions. To fix this, every policy now gets checked at the ASP; when reserving a segment, it is checked if that segment collides with a static object in the Grid Map by rasterising a line the width of the UAV between the two segment points.

While the aim for the framework was to be able to easily plug in new planners, WHCA*'s strength was to return partial paths, which the framework would believe is an incomplete path and throw a warning. Now the UAV state machine is set to return to `en_route` for more path if the path given did not get the UAV to its destination.

3.6 Summary

The implementation of the framework closely follows the architecture described in Chapter 2. The eight planning policies share the same base MAPF policy class. The reservation mechanism and the continuous collision detection are the principal engineering contributions of the implementation work. Validation was performed through unit testing, determinism checks, and end to end runs across all eight policies.

Chapter 4

Results, Evaluation and Discussion

This chapter reports the experimental evaluation of the simulation framework described in Chapters 2 and 3. Section 4.1 documents the experimental setup and the matrix of configurations. Section 4.2 presents the results on segment based reservation. Section 4.3 analyses the computational cost of the policies. Section 4.4 summarises supplementary experiments. Section 4.5 discusses the broader algorithmic findings. Section 4.6 outlines the limitations of the evaluation. Section 4.7 draws conclusions from the experimental data. Finally, Section **??** identifies directions for future work.

4.1 Experimental setup

The main experiment that took place was a sweep over the eight policies and the binary `allow_predicted_collisions` toggle introduced in Chapter 2, at two simulation durations and across five seeds. Each configuration was executed twice to verify determinism. Three supplementary experiments held the policy set fixed and varied one independent variable at a time. These were fleet size, obstacle density and the rate of delivery jobs. Table 4.1 summarises the experiment matrix.

Table 4.1: Experimental matrix.

Main sweep	100, 500	42, 53, 82, 97, 23	2	All 8	`allow_predicted_collisions`
Long run	1000	42, 53, 82	2	Greedy, A*	`allow_predicted_collisions`
Fleet scale	100	42	1	All 8	Number of UAVs
Obstacle density	100	42	1	All 8	Obstacle proportion
Spawn rate	100	42	1	All 8	Poisson rate λ

Five metrics are recorded per configuration. Completion rate is the proportion of jobs delivered within the simulation time limit. Average delivery time is the mean simulated elapsed time between job assignment and delivery (in seconds). Total collisions are recorded by the ASP through the continuous collision check. Minimum separation is the smallest distance between two UAVs observed during the simulation, in metres. Runtime is the real world time it took to run the simulation; it will be used as a proxy for computational cost.

Five seeds are the minimum required for a meaningful descriptive comparison. The results are reported as means across these seeds. The supplementary experiments use a single seed and therefore only observations. The framework's deterministic execution, verified empirically in Chapter 3 by rerunning every configuration with bit identical results, supports the reproducibility of every figure presented in this chapter.

4.2 Effect of the reservation mechanism

Figure 4.1 presents the comparison of the eight policies under both states of the `allow_predicted_collisions` toggle at sim_time = 500 s, averaged across the five seeds.

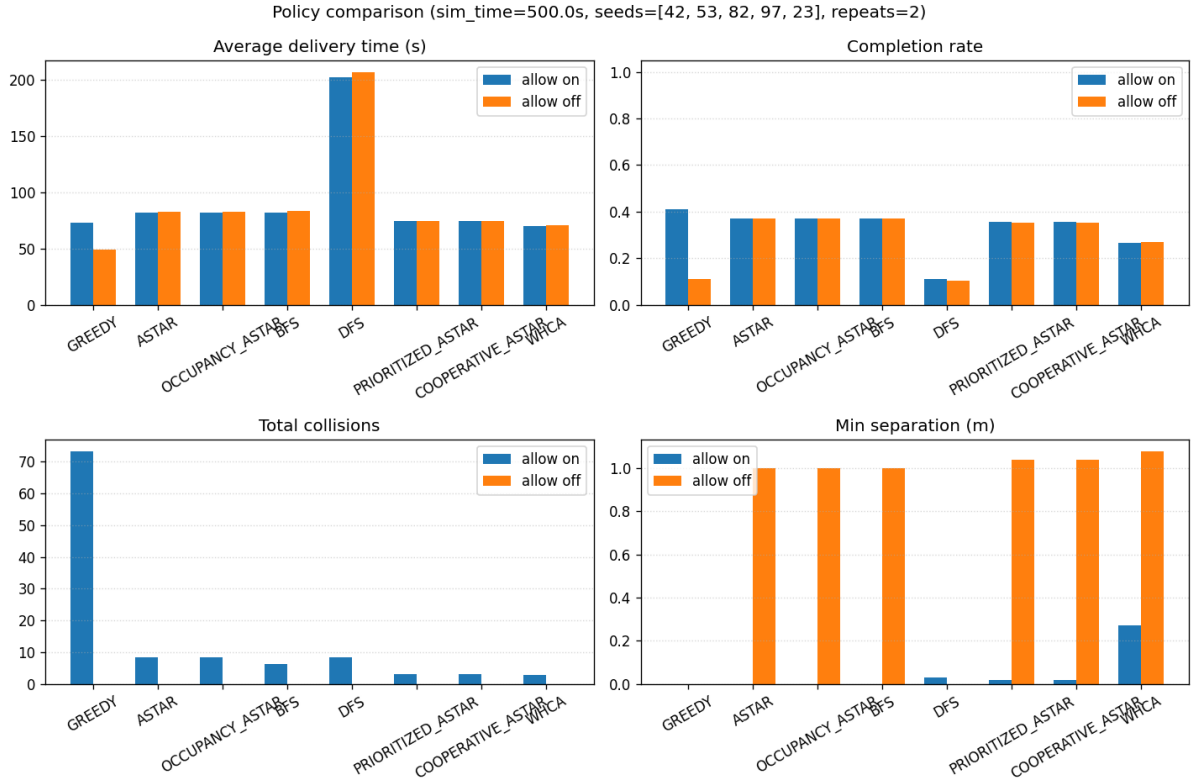

Figure 4.1: Policy comparison at sim_time = 500 s, seeds {42, 53, 82, 97, 23}, two repeats per configuration. Blue bars: `allow_predicted_collisions = True` ("allow on"); orange bars: `allow_predicted_collisions = False` ("allow off").

The total collisions graph shows the central finding that this project aims to demonstrate. With the reservation mechanism advisory (allow on), Greedy averages 73 collisions per run, A*, Occupancy A*, BFS, and DFS average between six and nine, and Prioritised A*, Cooperative A*, and WHCA* average three. But, when the mechanism is toggled, every policy had zero collisions. The reservation mechanism therefore constructively prevents UAV on UAV collisions.

The completion rate graph shows that the reservation mechanism causes a large fall in Greedy's completion rate, but almost no change for other policies (all other policies are essentially unchanged: A*, Occupancy A*, and BFS hold at approximately 0.37; Prioritised A* and Cooperative A* at approximately 0.35; WHCA* at approximately 0.27; and DFS at approximately 0.10). This is entirely down to the fact that no other policy will submit a reservation that intersects a static obstacle, which means that the ASP will reject the reservation and UAVs using the Greedy policy will fail, causing the UAV to escalate through `HOVER_WAIT` to `EMERGENCY`.

The average delivery time panel shows the same issue. Greedy's allow on delivery time of 73 s

drops to 50 s under allow off, reflecting that the few deliveries Greedy completes under enforcement are short, low conflict routes. Longer routes are rejected and never recorded. All other policies show essentially no difference in delivery time between the two states.

4.3 Computational cost

Figure 4.2 presents the real world runtime of each policy at sim_time = 500 s under both toggle states.

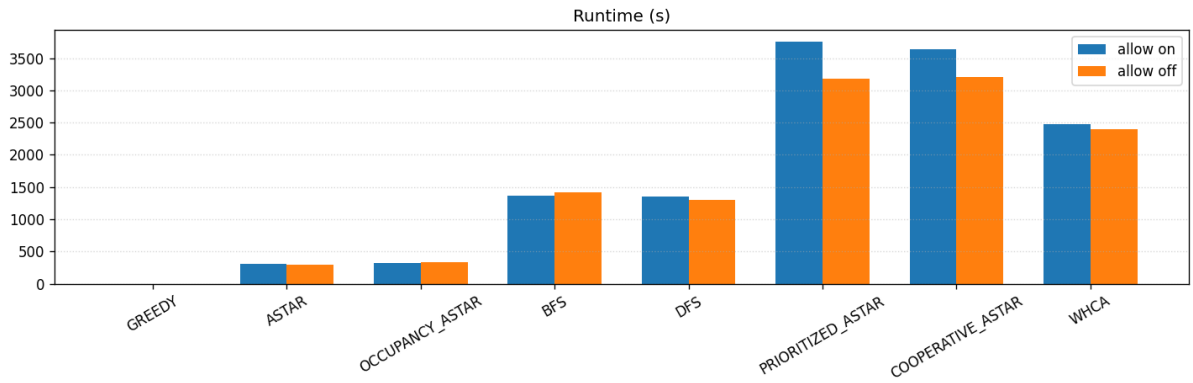

Figure 4.2: Wall-clock simulation runtime in seconds at sim_time = 500 s, seeds {42, 53, 82, 97, 23}, two repeats per configuration.

The runtimes span three orders of magnitude. Greedy runs in essentially zero time because it performs no search. Then it is informed search A* and Occupancy A* that run in approximately 300 s. After this, BFS and DFS run in approximately 1,300–1,400 s, reflecting their uninformed exploration. Then finally, reservation based policies are the most expensive; Prioritised A* and Cooperative A* run in approximately 3,200–3,800 s, and WHCA* in approximately 2,400 s. This therefore shows that the cost of reservation based coordination is approximately 10x that of A* and approximately 20x that of Greedy. WHCA* is an outlier in the pattern and it shows a lower runtime than the other reservation policies, which is likely due to its bounded horizon.

4.4 Behaviour under varying conditions

The three single seeded supplementary experiments examine fleet size, obstacle density, and demand intensity at sim_time = 100 s. They are reported as descriptive observations rather than statistically rigorous findings (due to the single seed and the short simulation time).

Varying fleet size between small, medium, and large produces essentially identical completion rates of approximately 0.14 for every policy (except DFS due to its massively inefficient nature). Completion rate is therefore not limited by fleet capacity, therefore adding more UAVs does not produce more deliveries. The bottleneck is elsewhere in the system; I believe it to be both depot takeoff and landing and also gate capacity. Despite no decrease in delivery

time with fleet size, the runtime however does increase with fleet size for all policies (such as WHCA* going from 45 s to 78 s). This is an expected result as more UAVs mean that more searches or routes are needed to be generated. Other than WHCA* and BFS, other policies show increases that are approximately linear to the number of UAVs.
In an obstacle heavy world, the same pattern is produced (albeit at a smaller scale). Completion rate is reduced to approximately 0.06 across policies (from 0.14–0.40 in the default), reflecting that higher obstacle density makes more routes infeasible.

4.5 Algorithmic findings

Three findings of broader interest emerged from this data.
The first is bounded horizon completability. WHCA*'s completion rate was consistently lower than Prioritised A* and Cooperative A* across the experimental matrix (approximately 0.27 versus approximately 0.35 given the sim_time = 500 s), despite all three sharing the same underlying search. This is the cost trade off with WHCA*: if a complete route requires too many time steps, WHCA* cannot find it.

The second is the trade off. A* (decoupled, no time expansion) and Prioritised A* (coordinated, time expanded) achieve similar completion rates (approximately 0.37 and 0.35 respectively) but exhibit substantially different collision behaviour under allow on with A* making up 3x more collisions. Therefore the reservation mechanism has increased the safety of the policy without reducing its efficiency.

The third concerns replanning capacity for strict deconfliction. The fall of Greedy's completion rate from 0.41 to 0.11 by toggling reservation shows that strict deconfliction cannot be applied to policies that do not have the capacity to replan around rejected reservations. If a UAV's reservation is rejected and the policy can only give the same plan back, the UAV will fail. The binary toggle shows this in a way that the cooperative pathfinding literature typically does not.

4.6 Limitations

The main sweep uses five seeds, which is insufficient for statistical claims such as confidence interval reporting and significance testing. Furthermore the supplementary experiments use a single seed and are reported as descriptive observations (due to the single seed and the short simulation time). The main simulation duration is short relative to typical UTM deployments and completion rates cluster around 0.35–0.40 across most policies, which Section 4.4 attributes to an infrastructure bottleneck rather than to policy capacity. However in the real world, even the high end 40% completion rate is not acceptable to be relied on by business, governments and people. Furthermore, the Conflict-Based Search (CBS) family [16] and the LaCAM family were not implemented along with active collision avoidance methods such as the Velocity Obstacle (VO) family.

4.7 Conclusions

The project set out to evaluate the comparative performance of automated airspace management policies for ASP mediated urban UAS traffic, with particular attention to the safety and efficiency trade off that distinguishes one policy from another. The work framed the coordination problem as Multi Agent Path Finding. Three principal claims are supported by the experimental data.
First, the segment based reservation mechanism is the key driver for safety within the framework. The reservation mechanism is the only "policy" that consistently reduced collisions to zero. It was proved empirically through the binary testing of the `allow_predicted_collisions` toggle. The mechanism's effectiveness is independent of the underlying search algorithm so long as the algorithm can replan effectively. This is the contribution of strategic deconfliction in the sense of Silver [17] also isolated empirically.
Second, the computational cost of strategic deconfliction is real but bounded. While reservation based policies are safer they come at over a 10x increase in runtime compared to A*. At scale this cost would be non trivial for an ASP to bear. However on the other hand using the reservation mechanism does not increase the runtime of non reservation based policies. Therefore this tool would be invaluable for an ASP to have in its toolkit.
Third, the principal limitation of this evaluation is the seed budget. Five seeds support descriptive comparison and the qualitative findings reported above, but not confidence interval reporting or significance testing. The completion plateau observed across policies is a structural limitation indicating an infrastructure bottleneck of the policies. This is critical to the results as it suggests that the framework needs rethinking to produce non limited results.

4.8 Future work

If given more time to work on this project, there are several directions in which I would extend this work.

I would have expanded the algorithmic coverage to include algorithms such as CBS [16] and LaCAM, which are both widely used in cooperative literature. This would allow for a more comprehensive comparison between different planning mechanisms. I would also include the D* lite algorithm, which is a single agent replanning algorithm that could be used in a decoupled policy to give it the capacity to replan around rejected reservations which in theory would be the best pairing with the reservation mechanism.

I would have implemented active collision avoidance methods such as the Velocity Obstacle (VO) family, which are widely used in MAPF literature. While the mechanism is partially present in the current framework, it would provide a comprehensive mechanism for collision avoidance that could be compared against the reservation mechanism.

I would have extended the simulations to run for extended times and with more seeds to allow for statistical claims to be made. This would allow for more empirically rigorous conclusions to be drawn from the data. Along with this the infrastructure bottleneck would need to be

addressed to allow for higher completion and more meaningful comparisons as policies do not hit the cap and plateau at around 0.35–0.40.

Fourth, I would have extended the reservation mechanism to have a more complex interaction with policies. For example instead of like now just hovering and waiting for a reservation to be accepted, the UAV could have a more complex behaviour such as replanning around a rejected reservation.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4 edition, 2022.

[2] M. Erdmann and T. Lozano-Pérez. On multiple moving objects. *Algorithmica*, 2(1–4): 477–521, 1987.

[3] European Union Aviation Safety Agency. EASA certifies first U-space service provider in the EU. Press release, European Union Aviation Safety Agency, 2025. Available from: https://www.easa.europa.eu/en/newsroom-and-events/press-releases/easa-certifies-anra-technologies-first-u-space-service-provider [Accessed 13 May 2026].

[4] Federal Aviation Administration. Unmanned aircraft system traffic management (UTM) concept of operations, version 2.0. Technical report, U.S. Department of Transportation, Federal Aviation Administration, Washington, DC, Mar. 2020.

[5] P. Fiorini and Z. Shiller. Motion planning in dynamic environments using velocity obstacles. *The International Journal of Robotics Research*, 17(7):760–772, 1998.

[6] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2): 100–107, 1968.

[7] F. Ho, R. Geraldes, A. Gonçalves, M. Cavazza, and H. Prendinger. Improved conflict detection and resolution for service UAVs in shared airspace. *IEEE Transactions on Vehicular Technology*, 68(2):1231–1242, 2019. doi: 10.1109/TVT.2018.2889459.

[8] F. Ho, A. Salta, R. Geraldes, A. Goncalves, M. Cavazza, and H. Prendinger. Multi-agent path finding for UAV traffic management. In *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 131–139, Montreal, Canada, 2019. IFAAMAS.

[9] F. Ho, R. Geraldes, A. Gonçalves, B. Rigault, B. Sportich, D. Kubo, M. Cavazza, and H. Prendinger. Decentralized multi-agent path finding for UAV traffic management. *IEEE Transactions on Intelligent Transportation Systems*, 23(2):997–1008, Feb. 2022. doi: 10.1109/TITS.2020.3019397.

[10] S. Koenig and M. Likhachev. D* Lite. In *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI)*, pages 476–483, Edmonton, Canada, 2002. AAAI Press.

[11] P. Kopardekar. Unmanned aerial system (UAS) traffic management (UTM): Enabling low-altitude airspace and UAS operations. Technical Report NASA/TM-2014-218299, NASA Ames Research Center, Moffett Field, CA, 2014.

[12] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. In *Proceedings of the Sixteenth International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 837–845, São Paulo, Brazil, 2017.

[13] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig. Searching with consistent prioritization for multi-agent path finding. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI)*, pages 7643–7650. AAAI Press, 2019.

[14] M. Phillips and M. Likhachev. SIPP: Safe interval path planning for dynamic environments. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 5628–5635, Shanghai, China, 2011. IEEE.

[15] D. Sacharny and T. C. Henderson. A lane-based approach for large-scale strategic conflict management for UAS service suppliers. In *Proceedings of the 2019 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 937–945, Atlanta, GA, 2019. IEEE.

[16] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.

[17] D. Silver. Cooperative pathfinding. In *Proceedings of the First AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 117–122, Marina del Rey, CA, 2005. AAAI Press.

[18] A. Stentz. Optimal and efficient path planning for partially-known environments. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3310–3317, San Diego, CA, 1994. IEEE.

[19] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, R. Barták, and E. Boyarski. Multi-agent pathfinding: Definitions, variants, and benchmarks. In *Proceedings of the Twelfth International Symposium on Combinatorial Search (SoCS)*, pages 151–158. AAAI Press, 2019.

[20] Team SimPy. SimPy: Discrete event simulation for Python. Software documentation, version 4.x, 2024. Available from: https://simpy.readthedocs.io/ [Accessed 13 May 2026].

[21] UK Civil Aviation Authority. Future of flight: BVLOS delivery model. Technical Report CAP 3038, UK Civil Aviation Authority, London, 2024.

Appendix A

Self appraisal

A.1 Critical self evaluation

From the perspective of the project's aims and objectives, the project was successful. The framework supports evaluating MAPF policies under controlled conditions. However, looking back at the project, there are limitations and trade offs worth acknowledging.

The first concerns scope. The decision to implement eight policies from uninformed search to reservation based search wasn't the initial path of the project. Early on, the plan was to have many more, including coupled search methods and reactive policies. However, as half finished policies built up, the simulation revealed to me the significance of reservation policies. This is when the narrower scope became more compelling. However, this meant that the project didn't explore all the key MAPF strategies, which was a trade off I thought was partially acceptable.

The second concern is the evaluation budget. Five seeds is the floor for descriptive comparison and supports the qualitative findings reported in Chapter 4, but it is genuinely insufficient for confidence interval reporting. This limitation was visible from early in the experimental phase but became binding only late, when full matrix runs at sim_time = 1000 s consumed more real world time than anticipated. In fact, to simulate 1 "second", the simulation could take 7 real world seconds using the worst case configurations. A more disciplined approach would have established that time would have been consumed by the evaluation, allowing for an earlier start to testing for evaluation and therefore more seeds to be used.

The third concerns the completion rate plateau outlined in Section 4.4. It identifies an infrastructure bottleneck upstream of the policies, almost certainly in gate or depot resource capacity, but the project does not fully address where it comes from or how to mitigate it. It is arguably the most significant finding from the evaluation, but is the least well understood and, given more time, should've prompted fixing and re evaluating all together.

A.2 Personal reflection and lessons learned

Some broader lessons emerged from seven months of development.

The first was choosing to go down the route of a discrete event simulation framework with deterministic execution. It sounded like a reasonable approach; it was only a month in that I realised that I had made a time stepped simulation. The decision to change the code to a discrete event driven approach was a significant investment. Not only did it require the rewrite of the core simulation loop, but it also required changing how planners interacted with the simulation, making each one more complex than they would have otherwise been. However, I don't fully regret that decision because going down that path led me to finding segment based reservation, which was a significant contribution to the project and also became an area of

interest I hadn't thought about before. This still doesn't mean I wouldn't have made a large architectural decision as early as I did.

Second, I found that similar research projects overlooked the complexities of designing a simulation framework, especially when it comes to the repetitive side of software development. Setting up GitHub Projects with milestones, using feature branches with code review against the main branch, running the pytest suite under GitHub Actions on every push, and writing tests for components that would otherwise have been informally checked. These are conventional industrial practices, and they cost time that could in principle have gone into more features. If I had stuck to using the testing throughout instead of letting it get out of date, bugs could be diagnosed more quickly because the tests around them established a known good baseline. I would adopt and stick to the same workflow on any future project of comparable scope without hesitation.

A more specific reflection is on the value of writing about the work concurrently with doing the work. Several of the findings reported in Chapter 4, bounded horizon completability in particular, became clear to me only when I attempted to describe them in writing. Treating drafting as a thinking tool rather than a final stage activity surfaced questions about the empirical results that I would otherwise have missed. Future projects would benefit from earlier and more frequent drafting cycles.

A final reflection is on risk management. The original proposal's risk mitigation plan identified the advanced MAPF policy as the descoping target if time ran short. In practice, the binding constraint was the time left to test the framework rather than implementation effort, and the descoping that did happen affected the seed budget rather than the policy set. The lesson is that risk plans constructed at proposal time tend to anticipate the wrong risks. A more useful approach next time would be to revisit and rewrite the risk plan at each major milestone rather than treating it as fixed.

A reflection of a different kind concerns working around disruption. During the project I sustained a head injury and was subsequently diagnosed with post concussion syndrome, which kept me away from the university for approximately a month and reduced my effective working capacity for some weeks after that. The most important lesson was the value of having a well structured project management system. Fortunately, I was up to date on my GitHub project and my future goals in the current and next milestone, along with the tasks needed to be done with the time and difficulty, allowed me to step back into the project with little downtime figuring out what to do next.

If I were to identify a single thing I would do differently, it would be to design the metrics infrastructure earlier. The current metrics system was almost all designed after the bulk of the code was written. Looking back, it would have been better to have it during development due to the key debugging role it can play.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

The project produced and evaluated a simulation framework. It did not produce a deployed system, collect personal data or process information under the data protection regulations. The framework does not interact with real airspace and so does not fall under the regulations of the UK Civil Aviation Authority [21] or the European Union Aviation Safety Agency [3]. Therefore legal exposure for the project itself is minimal. However, if the project was to be deployed, it would fall under the UTM regulatory frameworks such as the European U-space regulations [3]. One liability the project could have would be if this framework was used to justify legislative or operational decisions; however, the project does contain a detailed limitations section to mitigate this risk. Another potential issue is the use of open source software (SimPy, NumPy, pytest, matplotlib); however, this was mitigated by using software under permissive licences and using them in accordance with those licences.

A.3.2 Social issues

The wider social context for urban UAS operations is non trivial, even though the project itself is a simulation. It doesn't factor the effects of UAS traffic in dense urban areas, nor does it track the social impact of Unmanned Aerial Vehicles (UAVs) potentially always choosing a route near you as it is the most efficient. Furthermore, UAVs typically have sensors such as cameras that will make people feel surveilled even if no surveillance is intended.

On the flip side, the project has the potential to be beneficial in rural spaces by allowing for more efficient deliveries to hard to reach places such as delivering a defibrillator or supplies to a remote location. Furthermore, the project has the potential to reduce emissions by allowing for the use of typically electric UAVs for deliveries that may have been done through more carbon intensive means such as vans.

A.3.3 Ethical issues

The project did not involve participants from animals or humans and so the standard ethical review categories do not apply. However, there are some ethical considerations from the downstream effects of the project.

First, the evaluation could be used to justify operational decisions in real UTM systems where mistakes have safety consequences. For example, if someone was to copy or adapt the framework and find an unknown bug that made it unsafe, it could lead to a real world accident. To mitigate that, I have been open about the limitations of the evaluation.

Second, real urban airspace could potentially be populated with a range of devices from emergency medical drones to commercial deliveries, and hobbyist flights have different social value and a policy that performs well in aggregate may perform badly for the categories that matter most.

A.3.4 Professional issues

This project was conducted in accordance with the spirit of the British Computer Society (BCS) code of conduct. The project used professional engineering tools such as version control and branches in GitHub. The literature review and evaluation was conducted with integrity to distinguish the project's contributions from others' work.

Appendix B

External Material

The simulation framework, evaluation harness, and report were developed by the candidate. The following external materials were used as ready made components in their unmodified forms.

- **SimPy** (version 4.x, BSD licence) [20]. Used as the discrete event simulation core, as described in Section 2.1. UAV processes, job generators, and resource objects are implemented against the SimPy API; the simulation event loop is provided by SimPy.
- **NumPy** (BSD licence). Used for numerical operations on trajectory vectors and for continuous time geometric calculations within the ASP.
- **Matplotlib** (PSF style licence). Used for generation of the figures presented in Chapter 4.
- **pytest** (MIT licence). Used as the test runner for the unit and integration test suite described in Section 3.4.
- **Python 3.12 standard library**. Used throughout; no claim is made over any standard library functionality.
- **University of Leeds Final Report LaTeX template**. Used as the document template for this report, in accordance with the requirements published by the School of Computer Science.